

QARCHGAUGE: Architecture-Guided Modeling of Practical Quantum Advantage

Abstract—Problem. Practical quantum advantage is often discussed as if faster quantum hardware will solve many applied problems automatically. This is not enough for computer architects. A useful claim must identify which future-system lever actually changes the outcome: logical gate latency, shot parallelism, error-correction overhead, readout/control latency, algorithmic quality, or the native HPC baseline. Without that decomposition, a speedup number does not tell architects where a quantum system should improve.

Approach. We present QARCHGAUGE, a measurement and architecture-projection framework for this break-even test on a Top-20 TOP500 leadership-scale HPC system. QARCHGAUGE runs native baselines and quantum-circuit versions of the same workload with shared inputs and quality targets. It then converts measured circuit cost, quality gap, native runtime, circuit meta-data, and repeated-execution structure into an advantage threshold over architecture variables: execution speed, shot throughput, quality recovery, and non-kernel overhead.

Result. On the leadership-scale system, our expanded sklearn digits sweep runs 160 cases across class pairs, sample counts, PCA dimensions, depths, and seeds. Quantum kernels require a median $421.9\times$ speedup to match the native path; QNN/VQC requires $64.9\times$ but usually misses native quality. We then run a 3,552-case practical suite across ML, molecule, MaxCut, and HamSim workloads on up to 256 GPUs, plus a 104-case OpenFermion/PySCF molecule coverage gate. Median required speedups range from $3,071.0\times$ for HamSim to $287,045.6\times$ for MaxCut. The architecture implication is workload-specific: at 90% quality-gap recovery, $10^4\times$ projected execution improvement covers 54.9% of HamSim cases, while $10^5\times$ covers 57.1% of molecule cases; ML and MaxCut remain primarily quality-limited. These results do not claim quantum wins today. They show where future quantum architectures must spend effort before speed becomes useful.

I. INTRODUCTION

Quantum computing is often described through broad application promises: better machine learning, faster molecular modeling, improved optimization, and natural Hamiltonian simulation. For computer architects, the question is more concrete. Which future quantum-system resource should improve first: logical gate latency, two-qubit fidelity, shot throughput, readout/control latency, memory/communication near the controller, or algorithmic quality? A future quantum path is useful only if it beats a strong native HPC path on the same input, at the same quality, after all application overheads are included. Current quantum hardware is not yet large and reliable enough to answer this question directly for many practical workloads. HPC systems therefore play a second role beyond accelerating

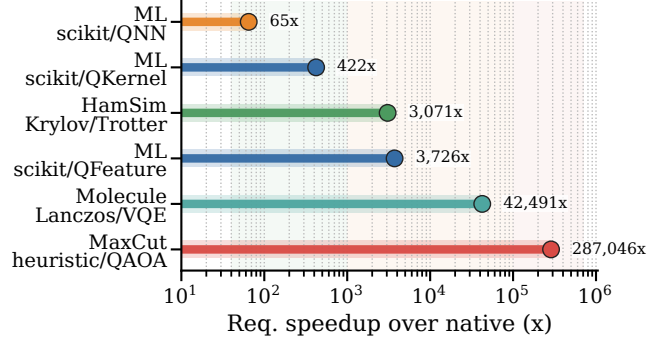


Fig. 1: Required speedup for each native/quantum pair. ML denotes classification, Molecule denotes molecular-energy estimation, HamSim denotes Hamiltonian simulation, and MaxCut denotes combinatorial optimization.

quantum-circuit simulation: they are the measurement instrument for modeling which architecture improvements would make a quantum-circuit application competitive.

Existing simulation systems provide the first part of this stack. Libraries such as cuQuantum execute quantum circuits on GPUs, while prior systems such as ScaleQsim and AURORA-Q improve simulation scalability on leadership-scale systems [1], [2]. These systems answer an important question: how can we simulate larger quantum circuits faster on HPC? However, simulator scalability is not the same as application-level quantum advantage. A simulator can be faster than another simulator, but the quantum-circuit application can still be slower than a native HPC application because encoding, circuit generation, repeated evaluations, measurement, optimizer steps, and postprocessing remain in the end-to-end path.

Figure 1 shows the comparison target of QARCHGAUGE. A circuit simulator is only one component of the quantum path. The full quantum path also includes application stages that do not disappear when the circuit kernel becomes faster. Three points follow. First, simulator speed is not application speed: measuring only cuQuantum time can hide encoding, measurement, optimization, and classical control. Second, native baselines are an architecture constraint because native HPC/ML applications already use mature software stacks and efficient kernels. Third, advantage is a design-space boundary: the goal is to invert today’s measurement and compute the required quantum gate time, shot throughput, error overhead, and quality recovery for future hardware.

TABLE I: Positioning of QARCHGAUGE relative to prior work.

Study/System	HPC sim.	Native baseline	Cost model	HW projection	Arch. bottleneck
ScaleQsim/AURORA-Q	✓				
cuQuantum	✓				
Benchmark suites			partial	partial	partial
Native baselines		✓			
QARCHGAUGE	✓	✓	✓	✓	✓

The same issue becomes stronger when the workload moves beyond one small ML example. Public claims about quantum computing often jump from one successful example to broad expectations such as better AI, faster drug discovery, or better optimization. A systems paper cannot validate such claims with one toy workload. It must cover representative application families and show the threshold for each family. For a practical workload, quantum advantage depends on several conditions being true at the same time: the quantum path must reach the target quality, the native baseline must be strong, the circuit must fit the available logical qubits, shots must be parallelized enough, and error overhead must not erase the speedup. QARCHGAUGE therefore reports an advantage frontier rather than a single yes/no answer.

Many previous studies focus on one part of this stack. Table I summarizes the gap. ScaleQsim and AURORA-Q improve quantum simulation on HPC systems. cuQuantum provides high-performance GPU primitives. Benchmark suites such as SupermarQ, QASMBench, MQT Bench, and QAOAKit improve coverage and reproducibility [3]–[6]. Native ML frameworks provide strong classical baselines. QARCHGAUGE connects these pieces into an application-level architecture requirement model: it reports whether the next useful improvement should be faster logical operations, more shot parallelism, lower error overhead, better quantum output quality, or a different algorithm.

Key Idea. QARCHGAUGE treats practical quantum advantage as an architecture design-space problem. The advantage threshold is the point where the projected quantum-circuit path becomes faster than the measured native HPC and ML path under the same workload and quality target. For each workload, QARCHGAUGE records the cost of each stage and then projects the quantum hardware region where this condition holds. The output is not one global speedup number; it is a bottleneck label that says whether future systems should prioritize gate throughput, shot parallelism, error/quality recovery, host-control overhead, or a stronger quantum algorithm.

We present QARCHGAUGE, an architecture-guided quantum-advantage modeling and analysis study for HPC systems. QARCHGAUGE starts with a controlled ML workload: `sklearn` digits native ML baselines, quantum kernel classifiers, and QNN/VQC classifiers. This first workload validates the measurement pipeline. We then scale the study to a practical suite with ML, molecular energy estimation, MaxCut optimization, and Hamiltonian simulation. The evaluation reports workload-level thresholds, leadership-scale campaign behavior, workload deep dives, design diagnosis, and quantitative projection targets.

This paper makes the following contributions:

- **Architecture threshold model.** We define practical quantum

advantage as a same-task, same-quality break-even condition between a native HPC path and a quantum-circuit application path, then express the result as future-system requirements.

- **Logging framework.** We build a pipeline that records run-time breakdowns, quality, circuit metadata, GPU metadata, Slurm metadata, and repeat-timing evidence.
- **Practical workload evidence.** We evaluate controlled quantum ML and a suite covering ML, molecule, MaxCut, and HamSim workloads, including a 3,552-case large profile, OpenFermion/PySCF molecule fixtures, and sparse Krylov baselines.
- **Scale-out validation.** We run the large profile up to 256 GPUs and report weak scaling, fixed-work scaling, bundled allocation behavior, and repeat timing.
- **Architecture-region analysis.** We convert measured circuit costs into required execution speed, shot throughput, and quality-gap recovery, then classify whether each workload is speed-limited, quality-limited, shot-limited, encoding-limited, or native-dominated.

II. BACKGROUND

A. Quantum-Circuit Application Paths

Quantum-circuit applications map input data or problem instances into a sequence of quantum gates. In quantum machine learning, common paths include quantum feature maps, quantum kernels, and variational quantum circuits. A feature-map pipeline encodes classical features into circuit parameters and extracts observables or kernel values. A QNN/VQC pipeline additionally optimizes trainable circuit parameters through repeated circuit evaluations.

Repeated Execution. The key systems implication is repetition. A single circuit execution is rarely the full application. Training and evaluation require many samples, classes, optimizer steps, shots, and circuit variants. Therefore, the cost of a quantum application must include encoding, circuit construction, repeated simulation or execution, measurement, loss computation, and classical postprocessing.

Practical Application Families. QARCHGAUGE treats the first ML workload as a calibration workload, not as the whole target. Practical quantum claims usually appear in four application families. First, quantum ML claims include classification, kernel methods, and variational classifiers. Second, chemistry and drug-discovery claims include molecular energy estimation, Hamiltonian simulation, and candidate screening loops. Third, optimization claims include portfolio, routing, scheduling, and QAOA-style objective minimization. Fourth, scientific simulation claims include quantum dynamics and materials workloads. These families have different native baselines and different quantum costs. A useful advantage model must therefore report a threshold per workload family, not one global number for all quantum computing.

Terminology. The project name is QARCHGAUGE, and the paper’s technical object is a *practical quantum-advantage threshold*. This is not a claim that current quantum hardware

wins on the measured applications. A threshold is the break-even condition under which the projected quantum-circuit application path becomes faster than the selected native HPC path while meeting the same quality target. We use *advantage region* for the set of speed and quality-recovery assumptions that satisfy this condition, and *architecture bottleneck* for the resource that prevents a case from entering that region. This distinction is important because a circuit simulator can be useful for measurement even when both the simulator and the projected near-term quantum path are far from application-level advantage.

B. Native Baselines and Threshold Model

Native HPC/ML baselines execute the same task directly on CPUs or GPUs. For the first QARCHGAUGE workload, the native path uses `sklearn` digits with PCA and classical classifiers such as logistic regression, MLP, and optionally SVM/RBF. These baselines are intentionally strong and simple: if a quantum-circuit model cannot beat a well-optimized native baseline at the same target quality, it does not satisfy the practical advantage threshold.

Leadership-scale GPU Platform and cuQuantum. The measurement platform is a Top-20 TOP500 leadership-scale HPC system with GPU nodes suitable for quantum-circuit simulation through `cuQuantum` and `cuStateVec` [7]. Our environment uses an existing `cuQuantum` installation and validates it through login-node smoke tests before any allocation-consuming job. Login-node tests are used only for correctness and pipeline validation. Performance experiments run through Slurm and record queue time, elapsed time, allocation resources, and charged node-hours.

Break-even Condition. For a fixed task, dataset split, target quality, and system budget, the native application path is:

$$T_{\text{native}} = T_{\text{input}} + T_{\text{preprocess}} + T_{\text{train}} + T_{\text{eval}} + T_{\text{postprocess}}. \quad (1)$$

The quantum-circuit path simulated on `cuQuantum` is:

$$T_{\text{qc_sim}} = T_{\text{input}} + T_{\text{encoding}} + T_{\text{circuit}} + T_{\text{cuQuantum}} + T_{\text{measure}} + T_{\text{postprocess}}. \quad (2)$$

The projected quantum hardware path is:

$$T_{\text{qhw}} = T_{\text{input}} + T_{\text{encoding}} + T_{\text{compile}} + T_{\text{execute}} + T_{\text{measure}} + T_{\text{error}} + T_{\text{postprocess}}. \quad (3)$$

The condition for projected quantum advantage is:

$$T_{\text{qhw}} < T_{\text{native}} \quad (4)$$

at the same target quality. An architecture study then asks which term must change to satisfy this inequality: native baseline strength, logical execution time, shot parallelism, error/control overhead, or quantum output quality.

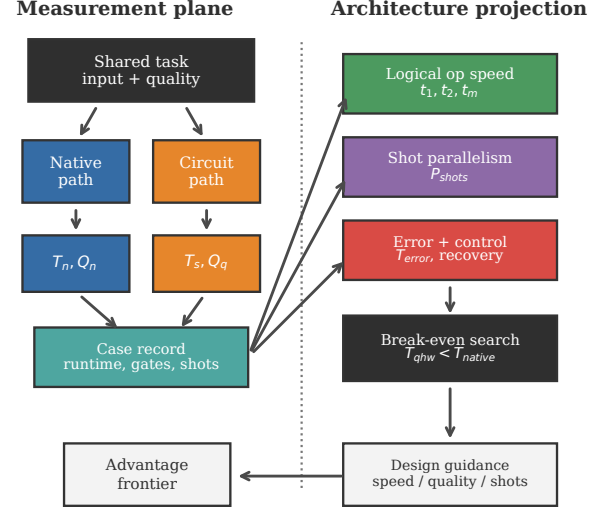


Fig. 2: Architecture and procedure of QARCHGAUGE.

III. QARCHGAUGE DESIGN

QARCHGAUGE answers one architecture question: which future quantum-system resource must improve before a quantum-circuit application can beat the native HPC application for the same task? It is not a new simulator, compiler, or quantum algorithm. It is a measurement and projection framework that keeps the input, quality target, native baseline, quantum-circuit path, and hardware projection in one auditable record.

A. Controlled Application Record

Shared input and quality. Figure 2 shows the core rule. Before any path runs, QARCHGAUGE creates one configuration record with workload family, instance ID, random seed, feature transform, native candidates, circuit family, circuit depth, shot setting, and quality metric. The native path and the quantum path receive this same record and append path-specific measurements. This prevents a quantum circuit from being compared against a different split, feature dimension, molecule, graph, Hamiltonian, or quality target.

Native path. The native path executes the best implemented non-quantum method for the same input. ML includes classical classifiers plus a production-native gate with PyTorch AMP CNN/MLP and XGBoost GPU-hist on the same measured digits configurations. Chemistry uses dense exact and sparse Lanczos/eigensolver baselines, optimization uses exact enumeration when feasible plus greedy, local-search, and annealing baselines, and simulation uses dense exact and sparse Krylov-style evolution. The threshold always uses the fastest valid native candidate that reaches the best native quality within a small tolerance, so adding a stronger native method can only make the quantum target harder.

Quantum-circuit paths. The quantum kernel path maps the same features into a quantum feature map, simulates the

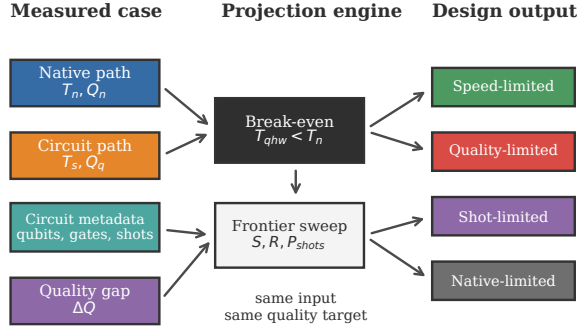


Fig. 3: Projection flow in QARCHGAUGE. Measurements are not the final answer; they are inverted into speed, quality, shot, and native-baseline requirements.

circuit evaluations needed to build a kernel matrix, and passes that matrix to a classical classifier. The QNN/VQC path maps the same features into a parameterized circuit and updates trainable parameters through a classical optimizer. QARCHGAUGE records application-level cost for both paths: encoding, circuit construction, cuQuantum execution, kernel or observable extraction, optimizer iterations, classifier-head time, accuracy, gate counts, shots, and repeated evaluations. This makes repeated kernel entries and hybrid optimizer loops visible instead of reporting only one fast circuit kernel.

B. Projection Interface

Simulator-hardware separation. The measured cuQuantum runtime is not a physical quantum latency. It executes the quantum-circuit application path, produces output quality, and records circuit metadata. The projection then prices the same case under future logical-operation speed, effective shot parallelism, error/control overhead, and quality recovery. Thus Python orchestration, GPU setup time, and simulator state-vector behavior are measurement-stack evidence, not future-QPU schedule terms.

Execution model. Given the measured metadata, QARCHGAUGE estimates projected quantum execution time as

$$T_{\text{execute}} = \frac{N_s(D_1 t_1 + D_2 t_2 + D_m t_m)}{P_{\text{shots}}} + T_{\text{error}},$$

where N_s is shot count, D_1 , D_2 , and D_m are one-qubit, two-qubit, and measurement depths, t_i are projected logical-operation times, and P_{shots} is effective shot parallelism. The overhead term is decomposed rather than hidden:

$$T_{\text{error}} = T_{\text{compile}} + T_{\text{map}} + T_{\text{route}} + T_{\text{decode}} + T_{\text{ms}} + T_{\text{io}} + T_{\text{cq}} + T_{\text{queue}}.$$

Likewise, useful parallel shots are bounded by the slowest system resource:

$$P_{\text{shots}}^{\text{eff}} = \min(P_{\text{device}}, P_{\text{decode}}, P_{\text{control}}, P_{\text{queue}}, N_s N_{\text{eval}}).$$

This form is the hook for architecture studies: a surface-code stack can plug in code distance, cycle time, decoder bandwidth,

and magic-state throughput, while a control-system study can change control and feedback terms.

Advantage frontier. QARCHGAUGE inverts the model instead of assuming speedup. It asks what logical speed, shot throughput, overhead, and quality recovery are required for $T_{\text{qhw}} < T_{\text{native}}$ at the same quality. A case is speed-limited when quality is close enough but runtime remains large; quality-limited when faster gates do not help until the output gap closes; shot- or encoding-limited when repeated evaluations or data movement dominate; and native-dominated when a stronger classical path moves the break-even target.

C. Evidence Discipline

Records and audits. Every run emits JSON records with dataset metadata, model metadata, runtime breakdowns, quality, circuit qubits, depth, gate counts, shots, GPU metadata, and Slurm metadata where available. Summarizers validate case counts and write CSV/JSON artifacts used by the figures; audit scripts then compare manuscript claims against those artifacts. Login-node tests are correctness gates only, while performance evidence must come from completed Slurm jobs.

Failure handling. Failed paths remain measurement results. If a native candidate fails, the record keeps the error and continues with other candidates. If a quantum path fails, the record keeps available circuit metadata and marks the case as non-advantaged. This rule matters at scale because removing difficult cases would overstate the frontier.

Workload scope. The controlled calibration workload is sklearn digits. The practical suite covers AI classification, VQE-style molecular energy estimation, QAOA-style optimization, and Hamiltonian simulation. These are the application classes where users often expect quantum advantage, but every row follows the same comparison discipline: same input, strongest auditable native path, measured quantum-circuit path, and explicit quality target.

IV. EVALUATION

This section follows the evaluation style of our previous systems papers: setup first, then the representative result, scale-out evidence, workload-specific detail, component diagnosis, architecture projection, backend ablation, and stability. The result is not a claim that quantum wins today. The result is a measured architecture model that shows how far each quantum-circuit application path is from its native HPC counterpart and which future-system resource must improve.

A. Evaluation Setup

Experiments run on a Top-20 TOP500 leadership-scale HPC system. Login-node smoke tests validate correctness, while performance experiments run through Slurm on GPU nodes with four NVIDIA A100 GPUs per node. The unit of comparison is a same-input application case: QARCHGAUGE measures a native HPC/ML path, measures a quantum-circuit application path on the simulator, records quality for both paths, and reports $T_{\text{qsim}}/T_{\text{native}}$ only when the quality target

TABLE II: Workload names and compared application paths.

Name	Meaning in this evaluation
ML-QNN/VQC	Machine-learning task solved with a parameterized quantum circuit; a classical optimizer learns circuit parameters, and quality is test accuracy.
ML-QKernel	Machine-learning task converted to a quantum feature map/kernel; many circuit evaluations build a kernel matrix consumed by a classical classifier.
ML-Feature	Practical-suite ML path; a quantum feature circuit transforms data and a softmax head performs classification.
Molecule-VQE	Molecular-energy task; native HPC uses exact diagonalization or sparse Lanczos, while the quantum path uses a VQE-style ansatz circuit.
MaxCut-QAOA	Combinatorial optimization task; native HPC uses exact, greedy, local-search, or simulated-annealing solvers, while the quantum path uses a QAOA-style circuit.
HamSim-Trotter	Hamiltonian-simulation task; native HPC uses dense or sparse Hamiltonian time evolution, while the quantum path uses a Trotterized Hamiltonian simulation circuit.

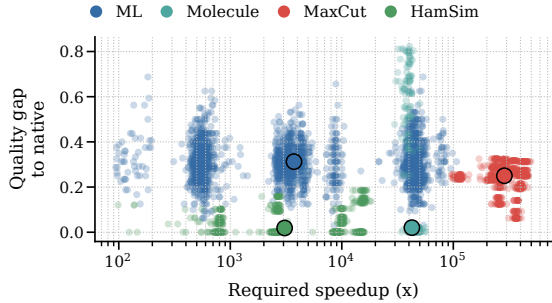


Fig. 4: Large practical-suite landscape over 3,552 cases. The plot keeps both axes that matter for advantage: required speedup and quality gap. A workload is not close to advantage unless it moves left in runtime and down in quality gap.

is interpretable. The native path is never another quantum simulator.

The completed campaign contains a 160-case controlled ML calibration, a 3,552-case practical suite, 116-case and 104-case molecule/HamSim coverage gates, weak and fixed-work scaling from 1 to 256 GPUs, a 7,104-case larger-workload gate on 256 GPUs, and 12 warmup-separated repeat trials. Performance evidence comes from Slurm jobs with accounting records, raw JSON counts, and completed exit status.

B. Workload-Level Thresholds

The main result starts at the workload level because practical advantage is not a simulator-versus-simulator comparison. The large practical suite covers ML classification, molecular energy estimation, MaxCut optimization, and Hamiltonian simulation. Each row compares a native HPC/ML implementation with a quantum-circuit application path under the same input and quality metric.

Figure 4 shows the representative application-level threshold landscape beyond the controlled digits sweep. The main practical-suite result uses the 3,552-case profile on 128 GPUs. ML selects among six native classifiers, MaxCut selects among exact, greedy, local-search, and annealing baselines, molecule uses dense exact and sparse Lanczos candidates, and HamSim uses dense exact and sparse Krylov candidates. The quantum paths are executed through the `cuQuantum/cuStateVec` application pipeline.

As shown in Figure 4, native execution remains extremely fast: the median native path is 2.09 ms for ML, 0.170 ms for molecule, 0.024 ms for MaxCut, and 2.08 ms for HamSim. The simulated quantum-circuit path is roughly seven seconds per case. This creates median required speedups of $3,726.4\times$ for ML, $42,491.4\times$ for molecule, $287,045.6\times$ for MaxCut, and $3,071.0\times$ for HamSim. These numbers are not a final hardware claim; they are the measured input to the projection model.

The threshold spread is caused by two forces: the native side often finishes in microseconds to milliseconds, while the quantum-circuit path pays the application cost of encoding, repeated circuit evaluations, state-vector simulation, measurement extraction, and quality scoring. HamSim has the smallest median runtime threshold, molecule has a larger but often speed-dominated threshold, and MaxCut has the largest threshold because the native path is extremely fast while the current QAOA path also carries a quality gap. ML is split: some circuit paths have moderate runtime thresholds, but advantage still depends on closing the quality gap.

Observation 1: The practical-suite gap is workload-specific. Native paths often finish in microseconds to milliseconds, while the simulated quantum-circuit path takes about seven seconds per case. The architecture question is therefore not whether the simulator is slow; it is whether future quantum systems should buy speed, quality recovery, shot parallelism, or lower control overhead for each workload family.

C. Scaling Methodology and Leadership-Scale Runs

The large-profile suite contains independent application cases, so the scaling study measures campaign throughput rather than distributed single-circuit simulation. This distinction is important: QARCHGAUGE uses Perlmutter to generate a large evidence set for advantage modeling, not to claim that a future QPU should behave like a multi-GPU state-vector simulator.

Weak scaling. Figure 5 shows weak scaling for the application sweep. Each GPU receives roughly 28 cases, so total work grows with GPUs. The comparable-profile sweep reaches 8–256 GPUs, while completed 1-GPU and 4-GPU pilot points show the low-end launch behavior.

As shown in Figure 5, all comparable weak-scaling jobs completed with exit code 0:0. The generated raw JSON counts match the expected case counts: 224, 448, 896, 1,792, 3,552, and 7,104. No failed case, traceback, or timeout was recorded in the task logs. Total throughput grows with GPU count while the per-GPU rate stays close to stable.

The sweep scales in this regime because cases are independent and each GPU receives a similar amount of work. This validates the evidence-generation method: Perlmutter increases application-case throughput rather than hiding a single-circuit communication bottleneck behind more GPUs.

Strong scaling. Figure 6 shows fixed-work strong scaling. Every full run executes the same 3,552 cases from 16 to 256 GPUs; the 1–8 GPU points are normalized from completed pilot/partial runs and marked separately.

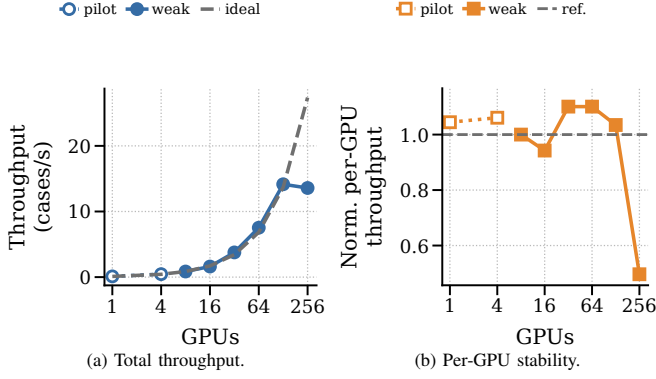


Fig. 5: Weak scaling with low-end pilot points. Hollow markers show completed 1–4 GPU pilot evidence; solid markers show the comparable weak-scaling suite from 8 to 256 GPUs. Total throughput grows with GPU count while normalized per-GPU throughput remains close to the 8-GPU reference.

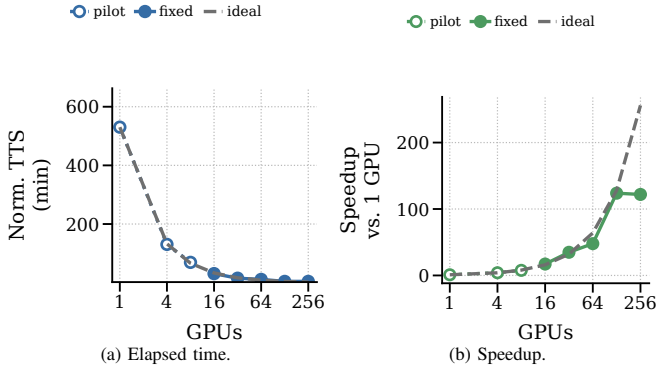


Fig. 6: Strong scaling of the fixed 3,552-case application suite. Hollow markers normalize completed 1–8 GPU pilot/partial runs to a 3,552-case equivalent; solid markers are direct fixed-suite runs. The measured full run reduces time-to-solution from 30 minutes 55 seconds on 16 GPUs to about 4 minutes 20 seconds on 128–256 GPUs.

As shown in Figure 6, the fixed-work result reduces elapsed time from 30 minutes 55 seconds on 16 GPUs to 4 minutes 17 seconds on 128 GPUs. The 256-GPU run completes in 4 minutes 21 seconds, nearly identical to the 128-GPU result.

The plateau is not distributed single-circuit MPI synchronization: every Slurm task owns independent cases on one A100. At 128 GPUs the sweep has about 28 cases per GPU; at 256 GPUs it has about 14. With a seven-second per-case floor, stragglers, Python orchestration, and task-local file output dominate before more GPUs can help.

Larger workload gate. Figure 7 shows the larger-workload gate. The separate 256-GPU run doubles the completed case count to 7,104 and checks whether time and quality remain interpretable at that scale.

As shown in Figure 7, the 256-GPU larger run completes 7,104 cases in 514 seconds with empty stderr logs. The workload-level quality gaps remain stable relative to the 3,552-case profile.

The larger run adds more independent application cases rather than changing the circuit model, strengthening coverage while preserving the same interpretation: runtime and quality must both move before any workload can enter the projected

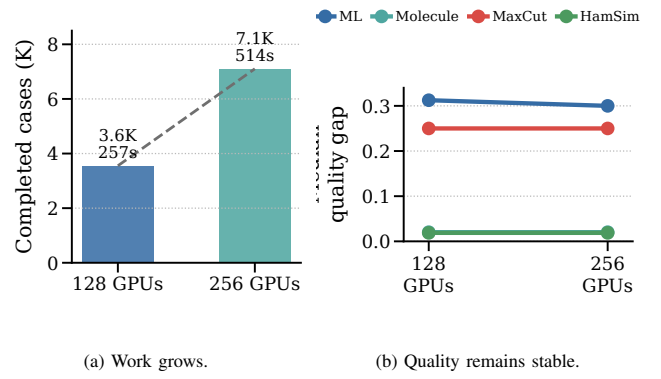


Fig. 7: Larger-workload gate on 256 GPUs. The 256-GPU run completes 7,104 cases in 514 seconds with empty stderr logs. The larger run preserves the workload-level quality interpretation, so the extra cases increase evidence coverage rather than changing the advantage conclusion.

advantage region.

Observation 2: More GPUs help until the application sweep runs out of useful task granularity. Weak scaling remains healthy through the 8–256 GPU comparable profile, but fixed-work strong scaling saturates after 128 GPUs. For this modeling workflow, architecture effort should improve per-case circuit execution and scheduling granularity, not only allocate more GPUs.

D. Workload Deep Dives

The workload-level result is the representative picture. This subsection gives the workload-specific evidence behind that picture, in the same style as ScaleQsim’s diverse-circuit analysis: each workload is treated as a separate case with its own baseline boundary and interpretation.

1) Controlled ML: scikit/QKernel and scikit/QNN: The controlled benchmark is the expanded sklearn digits sweep. It varies class pair, sample count, PCA dimension, circuit depth, and seed. The digits circuits use one qubit per PCA feature dimension, so the controlled sweep uses 4, 8, 12, and 16 qubits.

The native path is the fastest valid classifier among softmax regression, MLP, linear ridge, RBF kernel ridge, kNN, and nearest centroid on the same PCA features and split. QKernel maps the same features into a quantum feature map, simulates the circuit evaluations needed to build a kernel matrix, and passes that matrix to a classical classifier. QNN/VQC maps the same features into a parameterized circuit and updates trainable parameters through a classical optimizer.

Figure 8 shows the controlled ML result with both axes required for an advantage claim: runtime threshold and quantum-model quality.

As shown in Figure 8, native ML remains strong across the workload, with native accuracy between 0.9219 and 1.0000. Quantum kernels sometimes reach high quality, with median accuracy 0.8750, but the measured circuit path still requires hundreds of times faster projected execution to match the native path. QNN/VQC has a smaller required speedup in

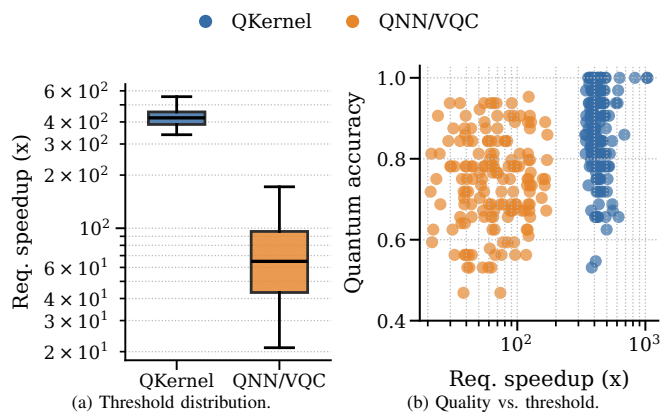


Fig. 8: Expanded digits results over 160 cases. Both quantum-circuit paths are compared against the selected native HPC/ML baseline on the same inputs. The quantum-kernel path often reaches useful accuracy but needs hundreds of times faster execution, while QNN/VQC has a lower threshold but usually lower quality.

many cases, but its median accuracy is 0.7500 and usually below the native baseline.

The two quantum ML paths fail in different ways. QKernel preserves more classification structure because it builds a kernel matrix over the feature map, but it must evaluate many circuits. QNN/VQC evaluates fewer circuits but is more sensitive to optimizer and ansatz quality. The easy pair, 0-vs-1, gives high median quantum-kernel accuracy at 0.9688, while harder pairs such as 3-vs-8 and 5-vs-8 reduce the median to 0.8125.

Observation 3: Controlled ML separates speed and quality bottlenecks. Quantum kernels need a median $421.9\times$ execution improvement to reach native time, while QNN/VQC needs only $64.9\times$ but usually misses native quality. A lower runtime threshold does not imply advantage if the quantum model misses the native target quality.

2) *ML Production-native Gate:* The main suite intentionally uses auditable small-task ML baselines because the quantum-circuit version is constrained to small feature dimensions and 4–16 qubits. To test the reviewer concern directly, we add a 32-case production-style ML native gate over measured digits configurations with PyTorch AMP CNN/MLP candidates and XGBoost GPU-hist. Figure 9 shows the resulting native-baseline gate and the profiling breakdown for this small-input ML setting.

As shown in Figure 9, the production candidates reach median quality 1.0000. Among production candidates, the AMP CNN is selected in 25 of 32 cases, XGBoost in five, and the AMP MLP in two. When these candidates are combined with the previous suite baselines, however, the selected native path remains the previous microsecond baseline in 24 cases and switches to the AMP CNN in eight. The median threshold changes from $8,876.8\times$ to $8,601.6\times$; the production-only threshold is $49.3\times$.

The GPU models are accurate but overhead-heavy for 8×8 digits, while the previous native baselines finish in microseconds on the same quantum-feasible input size. A larger

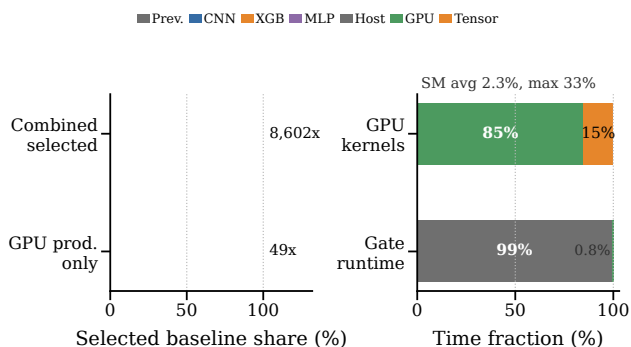


Fig. 9: ML production-native and profiling gate. PyTorch AMP and XGBoost candidates reach native-level quality, but the small digits task remains dominated by previous microsecond baselines and by host orchestration in the profiled GPU gate.

ResNet-style ImageNet baseline would measure a different problem scale rather than the required same-input comparison.

3) *Molecule and HamSim Coverage:* We run a separate accept-profile gate to strengthen the two application families that are easiest to criticize as toy baselines: molecular energy estimation and Hamiltonian simulation. The molecule gate includes OpenFermion and PySCF-generated H_2 , LiH, and H_2O active-space Hamiltonians, in addition to the built-in minimal H_2 and molecular-chain surrogate. The HamSim gate compares dense exact evolution with sparse Krylov evolution on TFIM and Heisenberg Hamiltonians from 4 to 8 qubits.

The strengthened molecule and HamSim baselines execute correctly in the same bundled one-node run. The job completed 116 cases in 6 minutes and 5 seconds with exit code 0:0; each of the four A100 tasks completed 29 cases, and stderr logs were empty. Molecule reports a median $63,566.8\times$ required speedup over 56 cases, and HamSim reports $4,185.2\times$ over 60 cases.

The stronger native side changes the interpretation because it prevents the quantum path from being compared against a single dense toy solver. For molecule, both dense exact and sparse Lanczos/eigensolver candidates are evaluated for every case. For HamSim, sparse Krylov becomes the selected native method in 34 of 60 cases, especially at 6–8 qubits.

H_2 , LiH, and H_2O active-space gate. We then run the expanded molecule-only accept profile with the 4–8 qubit OpenFermion and PySCF fixture set. The molecular integrals and Jordan-Wigner Hamiltonians are generated once into JSON fixtures before the timed application comparison; each timed case starts from the same stored Pauli Hamiltonian for both native and quantum paths.

Job 55515248 completed 104 molecule cases on one GPU node in 5 minutes and 23 seconds with exit code 0:0, 104 raw JSON outputs, and empty stderr. The larger active-space subset includes LiH and H_2O at 6 and 8 qubits. LiH active-space cases require $21,847.9\times$ at 6 qubits and $821.5\times$ at 8 qubits; H_2O requires $21,613.0\times$ at 6 qubits and $788.0\times$ at 8 qubits.

Dense exact remains the selected runtime baseline because these fixtures are still small, but sparse Lanczos is the best-

quality native method in 62 of 104 cases. This result keeps the comparison honest: the molecule path is no longer only H_2 , but the native side still reflects the problem size actually executable by the quantum-circuit path.

4) *MaxCut-QAOA*: MaxCut is the hardest practical family in the current suite. The native side selects among exact enumeration, greedy assignment, local search, and simulated annealing, while the quantum path uses a QAOA-style circuit.

In the strong-native run, the selected optimization baselines are mostly greedy assignment, with exact enumeration selected when it is fastest at the best cut value. The large practical suite reports a median $287,045.6\times$ required speedup for MaxCut.

This number is high because the native path is extremely fast and the current QAOA grid has a large objective gap. MaxCut therefore behaves like a quality-first workload before faster quantum execution can matter.

E. Design Diagnosis: Where Must Architecture Improve?

The previous sections show the measured gaps. This subsection answers the design question that a raw runtime plot cannot answer: where should a future quantum computer improve? We map each workload to four levers: native-baseline strength, algorithmic quality, quantum execution, and control-stack overhead. Figure 10 first shows why these levers differ by workload instead of using one global speedup target.

As shown in Figure 10, median records are ML: 7.3K ops, 224 evals, 7.3s; molecule: 0.7K ops, 41 evals, 7.1s; MaxCut: 3.4K ops, 101 evals, 7.0s; and HamSim: 0.2K ops, one eval, 6.9s. The long tail is workload-specific: ML moves from $3.7K\times$ median to $48.5K\times$ at p90, MaxCut from $287.0K\times$ to $424.2K\times$, molecule stays high but tight at $42.5K\text{--}56.6K\times$, and HamSim reaches $18.4K\times$ at max.

The quantum-runtime IQRs are narrow relative to the family differences in gate counts because the current application pipeline has a large per-case floor. After that floor is removed, the operation mix shows different pressure points: ML and MaxCut pay repeated-evaluation pressure, molecule pays energy-estimation and two-qubit-operation pressure, and HamSim is closer to a direct logical-evolution path.

Native baseline strength. Figure 11 shows the native-baseline stress test as a slope view. The strong-native run completed as a one-node, four-GPU bundled allocation.

As shown in Figure 11, the job finished 190 cases in 6 minutes and 59 seconds with exit code 0:0, 190 raw JSON outputs, and empty stderr logs. Strengthening the native side increases the ML threshold by $6.6\times$ and the MaxCut threshold by $2.3\times$.

A future quantum architecture competes against the native software stack available at deployment time, not against a frozen toy baseline. This is why the same quantum-circuit measurements can imply a different advantage threshold when the native candidate set is strengthened.

Host and GPU profiling. Nsight Systems records 53 CUDA kernel rows in the PyTorch AMP gate, including 16 tensor-family kernels with `xmma`, `wmma`, `tensorop`, and `ampere_fp16` names. Tensor-family kernels account for

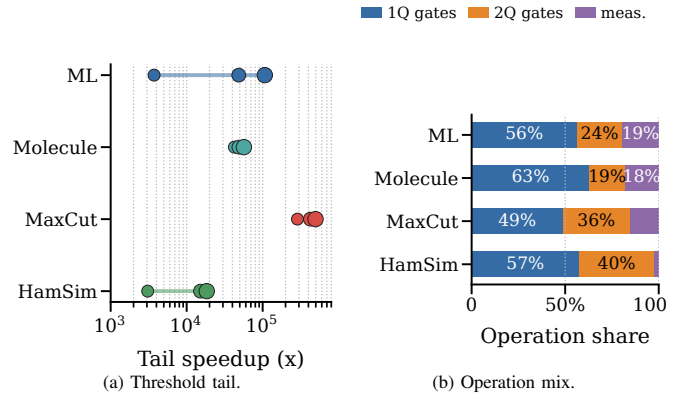


Fig. 10: Tail and circuit pressure by workload. The left plot shows median, 90th percentile, and maximum threshold pressure; the right plot separates one-qubit, two-qubit, and measurement pressure after the fixed simulation floor is removed.

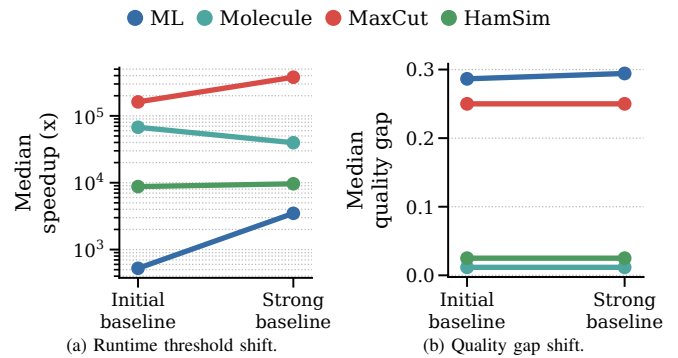


Fig. 11: Native baseline stress test. The slope view shows how each workload moves when the native path is strengthened, instead of hiding the shift inside grouped bars. Strengthening the native side makes the ML threshold $6.6\times$ larger and the MaxCut threshold $2.3\times$ larger than the initial native-baseline sweep.

15.2% of GPU kernel time, but GPU kernels occupy 0.8% of the run, or 48.4 ms of a 6.07 s profiled gate. The remaining 99.2% is host orchestration, data movement, Python framework overhead, and process/file activity around very small training jobs. `nvidia-smi dmon` shows 2.3% average SM utilization, a 33% peak, and no memory-bandwidth saturation signal. Nsight Compute/Roofline counter collection was attempted, but the driver reported an unavailable profiling resource; the artifact records the error instead of fabricating Tensor-Core utilization.

Workload diagnosis. Figure 12 classifies the 3,552-case large run using the same family tolerances as the projection frontier. This is the paper’s direct answer to “where must the architecture improve?”

As shown in Figure 12, ML is quality-limited in all 2,048 cases, and MaxCut is quality-limited in all 768 cases. Molecule has 48 speed-limited cases and 176 quality-limited cases. HamSim splits evenly, with 256 speed-limited cases and 256 quality-limited cases.

ML is quality-limited because the quantum feature path does not match the selected native classifier accuracy; its primary lever is algorithmic quality and hybrid-loop overhead. MaxCut is quality-limited because the current QAOA grid has a large

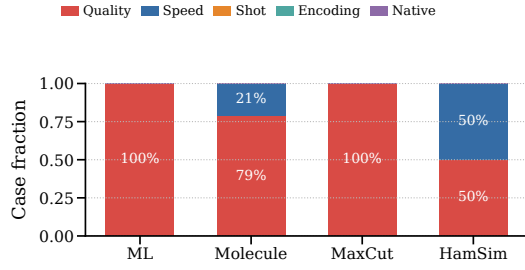


Fig. 12: Primary bottleneck taxonomy for the strong-native practical suite. ML and MaxCut are quality-limited in the current circuit configurations. Molecule is mostly speed-limited, while HamSim splits between speed and quality limits.

objective gap; its primary lever is ansatz/search quality before gate speed. Molecule needs both execution speed and better energy accuracy. HamSim’s useful region is the clearest target for logical-operation speed, shot parallelism, and control-stack overhead.

Observation 4: The design diagnosis separates the improvement targets. ML and MaxCut are not mainly waiting for faster gates; they need quality-preserving encodings, better ansatz/search structure, and lower hybrid-loop overhead before speed dominates. Molecule and HamSim are closer to execution and shot-throughput bottlenecks, but their useful region still depends on quality recovery and native-baseline strength.

F. Design Projection: How Much Improvement Is Enough?

Section IV-E gives the location of the bottleneck; this subsection gives the required amount of improvement. The practical-suite result gives one speed threshold per case, but a speed-only threshold is incomplete. A practical advantage claim also needs comparable output quality, enough shot/evaluation throughput, and a control stack that does not reintroduce a serial floor. We therefore convert each measured case into an advantage frontier over projected quantum execution speedup and quality-gap recovery, then instantiate the same records under a concrete shot/control sensitivity model. Figure 13 shows the primary frontier at 90% quality-gap recovery.

For a projected quantum speedup S and quality-gap recovery R , a case enters the advantage region only if

$$S \geq \frac{T_{qsim}}{T_{native}} \quad \text{and} \quad (1 - R)\Delta Q \leq \epsilon_w,$$

where ΔQ is the measured quality gap in the workload’s natural units and ϵ_w is the workload tolerance. The recovery parameter R is not a measured error-mitigation result; it is a requirement axis.

As shown in Figure 13, HamSim is the closest family: with 90% quality-gap recovery, a $10^4 \times$ projected execution improvement covers 54.9% of measured HamSim cases. Molecule needs a larger runtime shift: at the same recovery level, $10^5 \times$ covers 57.1% of molecule cases. ML shows the opposite behavior. A $10^5 \times$ speed improvement covers only 9.8% of ML cases at 90% recovery, but almost all cases if the

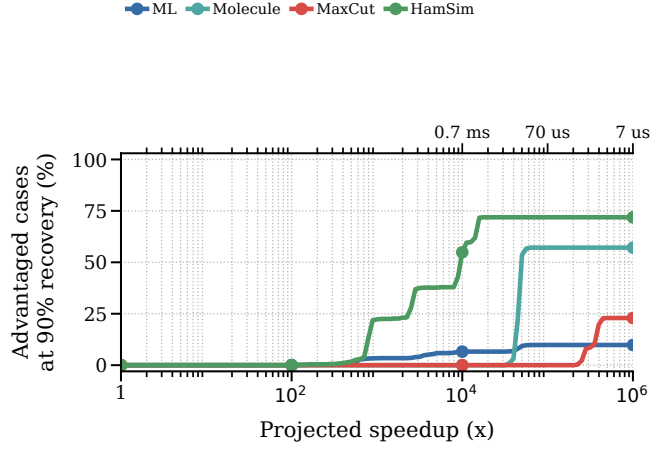


Fig. 13: Advantage frontier at 90% quality-gap recovery. The curves show the fraction of measured cases in each workload family that would enter the advantage region as projected quantum execution speedup improves. The secondary axis maps representative speedups to effective quantum execution time using the measured seven-second circuit path.

quality gap is fully closed. MaxCut is the hardest family in the current suite; even $10^6 \times$ speedup covers only 22.9% of cases at 90% recovery and requires full quality recovery to cover all cases.

The frontier should be read as an architecture target sheet rather than a simulator score. HamSim points first to logical-operation latency, measurement latency, and shot scheduling. Molecule needs those speed levers plus energy-quality recovery. ML and MaxCut require algorithmic-quality recovery before a gate-speed investment can translate into application advantage. The FT-stack sensitivity below then asks whether shot parallelism and decoder/control overhead let the speed target survive in a concrete system model.

Tolerance sensitivity. Figure 14 sweeps tolerance multipliers without rerunning GPU jobs. The projection uses strict family tolerances: 0.02 for ML and MaxCut and 0.01 for molecule and HamSim. These values are model inputs, not universal domain standards.

As shown in Figure 14, at 90% recovery, HamSim at $10^4 \times$ moves from 46.9% to 54.9% to 68.9% as the tolerance changes from $0.5 \times$ to $1 \times$ to $2 \times$. ML at $10^5 \times$ is much more tolerance-sensitive, moving from 0.8% to 9.8% to 83.8%. Molecule at $10^5 \times$ is speed-bound under these tolerances, staying at 57.1% until the tolerance becomes very loose.

Relaxing the tolerance helps only when quality is the active bottleneck. ML changes sharply because many cases are close to the runtime target but fail the quality gate. Molecule changes little because many cases are still runtime-bound. MaxCut remains hard unless both the speedup and quality budget are relaxed.

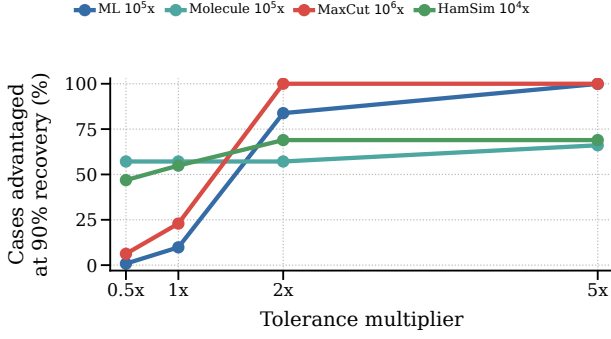


Fig. 14: Tolerance sensitivity of the advantage frontier. The same measured cases can be reinterpreted under stricter or looser quality budgets; this changes quality-limited ML and MaxCut much more than speed-bound molecule.

Shot and FT-stack sensitivity. Figure 15 instantiates the symbolic projection terms for a concrete subset-style check using every practical-suite case. The sensitivity uses the measured gate/evaluation metadata with an illustrative surface-code lower-bound stack: $d = 25$, $\tau_c = 1 \mu\text{s}$, $k_1 = 1$, $k_2 = 4$, $k_m = 1$, $5 \mu\text{s}$ decoder latency per circuit evaluation, and $50 \mu\text{s}$ of residual control/queue overhead per evaluation. Only the logical gate portion is divided by effective shot parallelism; decoder/control overhead remains serial in this lower-bound model.

As shown in Figure 15, runtime falls as effective shot parallelism increases, but the serial decoder/control floor prevents unbounded improvement. The speed axis is dimensionless, but it has a physical interpretation: since the measured circuit path is about seven seconds per case, $10^4\times$, $10^5\times$, and $10^6\times$ correspond to roughly 0.7 ms , $70 \mu\text{s}$, and $7 \mu\text{s}$ of effective end-to-end quantum execution per measured case before residual host and error terms.

This conversion is a normalization target, not a claim that gate speed alone reaches the target. A fault-tolerant instantiation reaches that budget only if logical cycle time, code distance, decoder bandwidth, magic-state throughput, controller scheduling, and effective shot parallelism jointly satisfy the decomposed model in Section III. The model therefore exposes when increasing P_{shots} stops helping.

Observation 5: The frontier converts measurements into design guidance. Faster logical operations and shot-level parallelism are most valuable for HamSim and part of molecule, but the FT-stack sensitivity shows that decoder/control overhead and finite evaluation parallelism create a floor. For ML and MaxCut, faster gates alone do not create advantage until quality and hybrid-loop overhead improve.

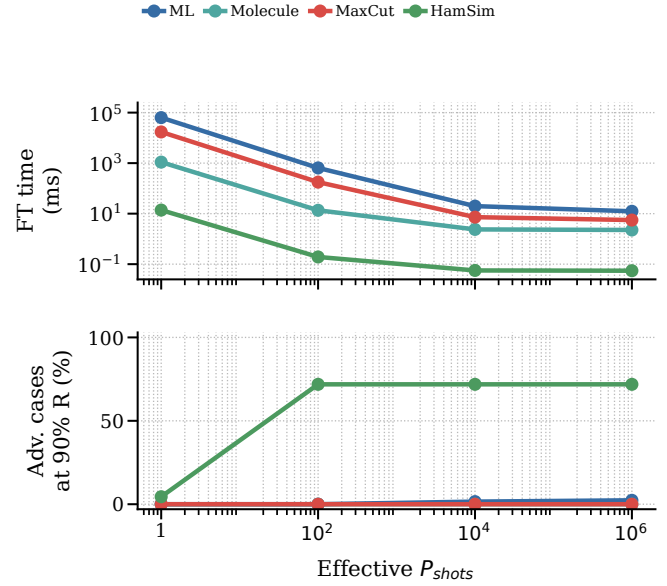


Fig. 15: Concrete FT-stack and shot-parallelism sensitivity. The same measured metadata is evaluated under a stylized surface-code stack. Runtime falls as P_{shots} increases, but the serial decoder/control floor and quality gate leave molecule, ML, and MaxCut constrained even when logical gate work is highly parallel.

G. Backend and Stability Gates

The production records use the `cuQuantum/cuStateVec` application pipeline so that every practical-suite case shares the same native path, quantum-circuit logic, simulator interface, and Slurm accounting path. A small backend-diversity gate confirms that other Perlmutter stacks are callable on sanity circuits: `cuQuantum/cuStateVec` passes in 3.18 s, `qsim CPU` in 4.50 s, `qsim GPU` in 0.55 s, `CUDA-Q cuStateVec` in 6.02 s, and `Qiskit Aer CPU` in 1.23 s; `Qiskit Aer GPU` is unsupported in the current device stack. This is not used as a performance ranking because the full application path includes encoding, repeated evaluations, optimizer/classifier logic, state extraction, quality scoring, and Slurm-recorded execution. The official sweep is also validated with bundled Slurm runs: one `salloc/srun` pilot completed in 6 minutes 54 seconds with 96 raw JSON outputs and empty stderr, and job 55516885 completed 12 warmup-separated trials in 58 seconds with exit code 0:0; the maximum quantum-runtime CV is 0.0400.

V. DISCUSSION AND THREATS TO VALIDITY

QARCHGAUGE does not prove current quantum advantage. It proves a computer-architecture point: once task, quality target, and native baseline are fixed, advantage becomes a measured design-space boundary that says whether future effort should buy faster logical gates, more parallel shots, lower error/control overhead, stronger native co-design, or better algorithmic quality.

The largest threat is an underpowered native baseline. We reduce it by selecting the fastest valid method among ML classifiers, exact/heuristic MaxCut solvers, dense/sparse molecular-energy solvers, and dense/sparse Hamiltonian-simulation solvers, then measuring the strong-native shift. The

same-input ML production-native gate with PyTorch AMP CNN/MLP and XGBoost reaches native-level quality but does not tighten the median threshold for tiny digits because GPU framework overhead dominates. Stronger natives on larger tasks, including deeper CNN/ResNet families, boosted trees, tuned MILP/domain heuristics, tensor networks, projector QMC, and CCSD(T)-class molecular references, can still move the frontier rightward; the method quantifies how much that stronger native path would raise the quantum requirement. The suite is not exhaustive: molecule cases use small active spaces, HamSim uses small qubit counts, QAOA uses fixed grids, and 4–16 qubit measurements are not deployment-scale extrapolations.

cuQuantum is instrumentation, not the future hardware stack. Real hardware adds logical gate time, readout, finite shots, feedback, mapping, routing, data loading, queueing, and error correction; hardware-specific studies should replace T_{error} and P_{shots} with code distance, cycle time, logical-error budget, magic-state throughput, decoder latency, controller bandwidth, and device concurrency. The FT-stack sensitivity is stylized, but it keeps decoder/control overhead serial and shows where more shot parallelism stops buying advantage. The same record can carry energy by comparing $E_{\text{native}} = N_{\text{GPU}} P_{\text{GPU}} T_{\text{native}}$ with a quantum budget that adds refrigerator, decoder FPGA/ASIC, control, and host power over T_{qhw} ; we leave those watts to facility-specific measurements. The scaling plateau is diagnosed from workflow structure, Slurm artifacts, and Nsight/dmon evidence showing 0.8% GPU-kernel time and 99.2% host orchestration on a small ML native gate. Nsight Compute counter collection failed because a driver profiling resource was unavailable, so we do not claim Roofline saturation. Bundled Slurm runs, 12 warmup-separated trials, zero failures, maximum quantum-runtime CV 0.0400, and artifact audits reduce timing and provenance risk.

VII. RELATED WORK

Computer architecture uses measurement and modeling to study immature systems [8]; QARCHGAUGE applies that discipline to quantum advantage. ScaleQsim, AURORA-Q, and cuQuantum improve state-vector simulation through distribution, GPUs, tiered memory, asynchronous movement, and optimized primitives [1], [2], [7]; benchmark suites such as SupermarQ, QASMBench, MQT Bench, QAOAKit, QED-C, and QCHALLENGE provide circuits, metrics, parameters, and application-aware tests [3]–[6], [9], [10]. Quantum kernels, variational classifiers, VQE chemistry, QAOA optimization, and Hamiltonian simulation are common advantage candidates [11]–[16], but each domain also has strong native baselines [17]. Fault-tolerant and resource-estimation work models code distance, cycle time, layout, magic-state factories, decoder latency, and physical assumptions [18]–[20]; hybrid runtimes add queue, fidelity, and classical-resource constraints [21], [22]. QARCHGAUGE is complementary: it supplies same-task native baselines, quality targets, measured circuit metadata, and shot/control sensitivity artifacts that those estimators can plug into.

VIII. CONCLUSION

We present QARCHGAUGE for same-input, same-quality quantum-advantage modeling. The controlled digits sweep requires $421.9\times$ for quantum kernels and $64.9\times$ for QNN/VQC; the practical suite contains 3,552 cases and shows that, with 90% quality-gap recovery, $10^4\times$ covers 54.9% of HamSim cases and $10^5\times$ covers 57.1% of molecule cases. The main lesson is to report an advantage frontier and architecture bottleneck taxonomy, not a yes/no slogan.

REFERENCES

- [1] C. Kim, E. Sohn, S. Kim, A. Sim, K. Wu, H. Tang, Y. Son, and S. Kim, “ScaleQsim: Highly Scalable Quantum Circuit Simulation Framework for Exascale HPC Systems,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 9, no. 3, Dec. 2025. [Online]. Available: <https://doi.org/10.1145/3771577>
- [2] C. Kim, A. Sim, K. Wu, H. Tang, Y. Son, J. Park, and S. Kim, “AURORA-Q: Asynchronous Unified Resource Optimizer for Quantum Simulation on HPC System,” Accepted to the IEEE International Conference on Distributed Computing Systems, 2026, accepted for publication.
- [3] T. Tomesh, P. Gokhale, V. Omole, G. S. Ravi, K. N. Smith, J. Vizslai, X.-C. Wu, N. Hardavellas, M. R. Martonosi, and F. T. Chong, “SupermarQ: A scalable quantum benchmark suite,” in *2022 IEEE International Symposium on High-Performance Computer Architecture*, 2022, pp. 587–603.
- [4] A. Li, S. Stein, S. Krishnamoorthy, and J. Ang, “QASMBench: A low-level quantum benchmark suite for NISQ evaluation and simulation,” *ACM Transactions on Quantum Computing*, vol. 4, no. 2, pp. 1–26, 2023.
- [5] N. Quetschlich, L. Burgholzer, and R. Wille, “MQT Bench: Benchmarking software and design automation tools for quantum computing,” *Quantum*, vol. 7, p. 1062, 2023.
- [6] R. Shaydulin, K. Marwaha, J. Wurtz, and P. C. Lotshaw, “QAOAKit: A toolkit for reproducible study, application, and verification of the QAOA,” in *2021 IEEE/ACM Second International Workshop on Quantum Computing Software*, 2021, pp. 64–71.
- [7] NVIDIA, “NVIDIA cuQuantum SDK Documentation,” <https://docs.nvidia.com/cuda/cuquantum/>, 2026, accessed July 3, 2026.
- [8] K. Skadron, M. Martonosi, D. I. August, M. D. Hill, D. J. Lilja, and V. S. Pai, “Challenges in computer architecture evaluation,” *Computer*, vol. 36, no. 8, pp. 30–36, 2003.
- [9] T. Lubinski, S. Johri, P. Varosy, J. Coleman, L. Zhao, J. Necaise, C. H. Baldwin, K. Mayer, and T. Proctor, “Application-oriented performance benchmarks for quantum computing,” *IEEE Transactions on Quantum Engineering*, vol. 4, pp. 1–32, 2023.
- [10] M. Erdmann, L. Karch, A. Awasthi, C. I. Jones, P. Bhardwaj, F. Krellner, J. Stein, C. Linnhoff-Popien, N. Kraus, J. P. Eder, S. Braun, and T. Liu, “Quantum computing—strategic recommendations for the industry,” 2026.
- [11] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, “Quantum machine learning,” *Nature*, vol. 549, no. 7671, pp. 195–202, 2017.
- [12] M. Cerezo, G. Verdon, H.-Y. Huang, L. Cincio, and P. J. Coles, “Challenges and opportunities in quantum machine learning,” *Nature Computational Science*, vol. 2, no. 9, pp. 567–576, 2022.
- [13] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow, and J. M. Gambetta, “Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets,” *Nature*, vol. 549, no. 7671, pp. 242–246, 2017.
- [14] E. Farhi, J. Goldstone, and S. Gutmann, “A quantum approximate optimization algorithm,” 2014.
- [15] I. M. Georgescu, S. Ashhab, and F. Nori, “Quantum simulation,” *Reviews of Modern Physics*, vol. 86, no. 1, pp. 153–185, 2014.
- [16] A. J. Daley, I. Bloch, C. Kokail, S. Flannigan, N. Pearson, M. Troyer, and P. Zoller, “Practical quantum advantage in quantum simulation,” *Nature*, vol. 607, no. 7920, pp. 667–676, 2022.
- [17] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

- [18] D. Litinski, “A game of surface codes: Large-scale quantum computing with lattice surgery,” *Quantum*, vol. 3, p. 128, 2019.
- [19] W. van Dam, M. Mykhailova, and M. Soeken, “Using azure quantum resource estimator for assessing performance of fault tolerant quantum computation,” 2023.
- [20] M. Webber, V. Elfving, S. Weidt, and W. K. Hensinger, “The impact of hardware specifications on reaching quantum advantage in the fault tolerant regime,” *AVS Quantum Science*, vol. 4, no. 1, p. 013801, 2022.
- [21] S. Pehlivanoglu, U. de Muelenaere, P. Kogge, and A. Sabry, “Qurator: Scheduling hybrid quantum-classical workflows across heterogeneous cloud providers,” 2026.
- [22] M. Tejedor, M. Grossi, C. Tüysüz, R. Rocha, and S. Vallecorsa, “Kubernetes-orchestrated hybrid quantum-classical workflows,” 2026.

AI USE

OpenAI Codex assisted with drafting, editing, code generation, artifact consistency checks, and LaTeX build debugging. The authors are responsible for the final technical content, experiments, figures, citations, and claims.